

MORF Language System for ESP32-S3 MCUs – Design Spec

Executive Summary: We propose **MORF (Mixture of Rational Forms)**: a radically device-native language engine that is **intentionally tiny** and **hand-structured**, not a compressed LLM. Instead of treating language as a huge learned embedding, MORF represents meaning as a **sparse composition of a few primitive “forms”** (semantic operations) and selectively fills in details. The system combines *hand-authored forms, explicit state and memory, deterministic logic, and minimal ML* to handle ambiguity or generate phrasing. This design exploits the ESP32-S3’s strengths (vector/DSP instructions, secure enclaves, Wi-Fi/BLE) while respecting its limits (few hundred KB SRAM, limited flash). We outline candidate MCUs, describe novel non-LLM architectures, and map high-level forms (like “increase”, “compare”, “association”) onto S3 vector/DSP ops. We specify a runtime execution model (stateful recurrence, salience-driven adaptive resolution), storage formats (form grammars, authored utterances), and a training pipeline (LLM-assisted corpus creation with human curation and targeted learning of gaps). We also cover evaluation metrics, security, OTA deployment, and a productization loop (grading, marketing, user feedback). Key milestones and a short list of forms/utterances are highlighted at the end.

1. Hardware Platforms (ESP32-S3 and Peers)

MCU	CPU & Freq	SRAM (Core)	Flash/PSRAM	Vector/DSP Accel	Wireless	Notes/Price
ESP32-S3 【6†L228-L232】	2×240 MHz Xtensa LX7	512 KB	Ext. SPI Flash (e.g. 4–16 MB), Ext. PSRAM configurable 【6†L228-L232】	Tensilica Vision P6 DSP (128-bit SIMD) 【25†L1-L4】 (CNN conv, GEMM, etc)	Wi-Fi + BLE5	Vector ISA; secure boot/AES-XTS 【22†L263-L268】
ESP32-P4 【10†L202-L205】	2×400 MHz RISC-V	1 MB I-RAM	SPI Flash, PSRAM	Vector DSP (PIE/QACC) (CNN) 【10†L202-L205】	Wi-Fi + BLE5	Faster CPU, more SRAM
STM32H755 (Cortex-M7)	2×480 MHz ARM Cortex-M7	2 MB + 512 KB	QSPI Flash, no external PSRAM	FPU & Ethos-U55 MicroNPU (4 MACs)	–	High perf, no Wi-Fi

MCU	CPU & Freq	SRAM (Core)	Flash/PSRAM	Vector/DSP Accel	Wireless	Notes/Price
K210 (Kendryte)	2×400 MHz RISC-V	~8 MB DDR	SPI Flash, PSRAM	CNN Accelerator (80 MACs)	–	Smart-IoT Vision chip
RP2040 (Pico W)	2×133 MHz Cortex-M0+	264 KB	QSPI Flash (2–16 MB)	No DSP SIMD (no MACs)	Wi-Fi	Low-cost, small
nRF5340	2×128 MHz Cortex-M33	512 KB	QSPI Flash	FPU, no vector DSP	BLE5	BLE, low-power

Trade-offs: *ESP32-*

S3 offers a balance of DSP acceleration and wireless connectivity. *ESP32-P4* boosts CPU/SIMD at higher clock (but is newer). *STM32H7* has more SRAM & an NPU but lacks Wi-Fi. *K210* has a neural engine but no comms. *RP2040* is low-cost but weak DSP. The choice depends on whether battery, cost, or features dominate; we focus on *ESP32-S3-class* as meeting AIoT needs.

Key Sources: Espressif's ESP32-S3 datasheet [【6†L228-L232】](#) [【22†L263-L268】](#) ; ESP-NN/ESP-DL performance benchmarks [【10†L277-L284】](#) [【24†L1-L4】](#) .

2. MCU AI Acceleration & Toolkits

The ESP32-S3 includes a Vision P6 DSP and vector ISA that accelerate typical NN primitives [25†L1-L4] . Espressif’s libraries exploit this: **ESP-NN** (CMSIS-NN port) and **ESP-DL** provide optimized kernels (Conv, GEMM (dense matmul), Add, Mul, pooling, activation, etc.) for 8-bit models [24†L1-L4] [10†L277-L284] . Benchmarks show ESP32-S3 conv layers running ~5–14× faster than naive C [10†L277-L284] . In practice, we will build MORF primitives to use these low-level kernels. For example, many forms reduce to dot-products or accumulation, which map to the DSP MACs or GEMM ops (8-bit/in16 accumulators). The ESP-IDF also provides an **ESP-DSP** library for vector math (FIR filters, statistics) and hardware intrinsics.

Memory: 512 KB on-chip RAM [6†L228-L232] is tight. We assume at least 4–8 MB external PSRAM (common on dev boards) for vocabulary tables, if needed. Flash (4–16 MB) stores code, models, and authored utterances.

Security: The S3’s built-in crypto (AES-XTS flash encryption, RSA secure boot, HMAC) and dual “World Controller” allow a trusted enclave [22†L263-L268] . We assume firmware and sensitive data (keys, profiles) are protected, and MORF operates entirely offline (no cloud dependency) for privacy.

3. Candidate Non-LLM Architectures

Rather than a monolithic neural net, MORF is a **hybrid system**. Candidate architectures include:

- **Rule-based / Expert Systems:** A set of finite-state or grammar rules for dialogue, possibly with lookup tables for slot values. Pros: tiny and interpretable; Cons: brittle and hard to scale if done fully by hand. MORF can use a lightweight rules layer for *highly reliable routines* (e.g. “turn volume down”) with deterministic parameters (x-forms, constraints).
- **Retrieval / Memory Network:** A structured memory of past examples and their actions. A new utterance is matched (e.g. by vector similarity) to memory and the corresponding response is reused. Pros: human-curatable; Cons: needs careful indexing in small state space.
- **Sparse Vector/Graph Recurrence:** Maintain a small state vector or graph; process input by combining state and input (via form operations) into a new state. Repeat across time. The “brain” is the transition operator on states rather than a fixed deep network. This resembles an RNN but with tiny state and operations drawn from domain primitives.
- **Neuro-symbolic mixture:** Represent some knowledge/syntax symbolically (like semantic templates) and use a small neural net to interpret ambiguous parts. For example, a neural “intent classifier” feeding into symbolic action logic.

MORF’s innovation is to blend these: the **primitive forms** (below) and the device’s firmware (acting as an “expert”) play the role of different modules. For example, “**TURN_DOWN_VOLUME**” need not be learned by MORF at all; it’s built from a form (excess intensity→decrease volume) applied to the audio volume parameter. The learning / neural part only covers “Where does volume control apply?” or “Who is speaking?”, not the math of volume.

4. Rational Forms (Primitives)

We propose a **small set of reusable computational primitives (“Rational Forms”)** that capture basic semantic/functional operations. Each form is implemented via simple math or logic (ideally using the S3’s vector/DSP ops). Forms can be **composed** to represent complex meanings. Example forms include:

- **Similarity/Association:** Compute dot-product or cosine similarity between vectors. (Implemented via `GEMM` or vector MAC.) E.g. form “semantically similar concept”.
- **Accumulation/Count:** Summation or average of a vector. (Vector add + divide.) E.g. form “aggregate signals” or “score sum”.
- **Comparison/Gating:** Compare two values (greater/less) and branch. (DSP has bitwise and saturating ops.) E.g. form “is X above threshold?” or “if condition then ...”.
- **Projection/Embedding Lookup:** Map an index or token to a vector (lookup in a small table). (Table in flash/PSRAM, indexing plus optionally a vector operation.)
- **Elementwise Transform:** Apply a nonlinearity (ReLU, sigmoid) or bitwise mask. (ESP-DSP has vectorized ReLU, etc.) E.g. form “normalize” or “boolean mask”.
- **Convolution/Correlation:** 1D sliding dot-product. (Use the Conv kernel with $1 \times N$ filters.) E.g. for matching a pattern in a sequence.
- **Competition/Pooling:** e.g. max or softmax among vector elements. (Max-pool kernel in ESP-NN.) E.g. form “pick strongest hypothesis”.
- **Memory Association:** Bind key→value pairs, and retrieve by similarity. (E.g. maintain a matrix M ; to recall, compute `state · M`). Uses GEMM.
- **Temporal Update/Prediction:** Combine previous state with new input (e.g. RNN-like `state = f(W · [state, input])`). (Small matrix multiply + add.)
- **Elementwise Add/Multiply:** Simple broadcast arithmetic (vector add, mul). (ESP-DL supports Add, Mul `[24†L1-L4]`.) E.g. form “combine features”.

These forms are **not bound to words**. For example, the form “Increase(Intensity)” could apply to *volume*, *brightness*, or *motor speed* depending on context. Each form’s implementation will use the S3’s fast integer operations (8-bit or 16-bit) to minimize cycles. We will store any learned parameters (e.g. small weight matrices, meaning prototypes) as int8 tensors so they fit ESP-NN/ESP-DSP.

Below is an illustrative table mapping some conceptual forms to S3 operations:

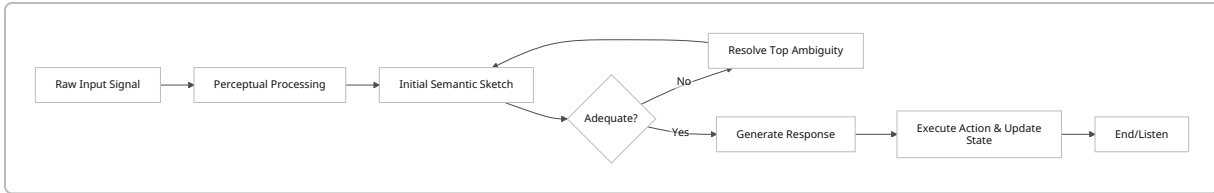
Semantic Form	Description	Implementation on S3
Similarity	Vector dot-product / cosine (are two things alike?)	<code>GEMM</code> or vector MAC (e.g. <code>vadd</code> / <code>vdot</code> instr.)
Association	Memory key-value recall (if X, then Y)	Matrix multiply (state × associative memory)
Accumulate (Sum)	Sum of features or signals	Vector sum/reduce (elementwise add + accumulator)
Comparison	Check if >/<; select branch	Vector compare (bitwise or subtract & test)
Gate/Threshold	Nonlinear gating (e.g. hard sigmoid)	Vector min/max or use LUT (small table)
Projection	Lookup or embed discrete input	Table lookup (flash/PSRAM) + optional vector op
Convolution/ Match	Slide a pattern window (1D conv)	1D Conv kernel (sliding dot-product) 【24†L1-L4】
Competition/ Pool	Find max or sum across candidates	Max-pooling or reduction (ESP-NN pool)
Elementwise Ops	+, −, × on vectors or scalars	<code>vadd</code> , <code>vsub</code> , <code>vmul</code> (ESP-DSP intrinsics)
Temporal Update	Combine old state & new info (small RNN step)	Small GEMM (dense layer) + add

We **hand-author** the high-level composition of these forms. For example, parsing “turn the music down” might invoke the grammar *AGENT “you” → TARGET “music” → ACTION “decrease intensity”*. The last part “decrease intensity” is mapped to the form sequence *GET(AudioIntensity) → COMPARE(>desired) → DECREASE()*.

Citation: The ESP-DL library explicitly supports **Conv, GEMM, Add, Mul** on ESP32-S3 **【24†L1-L4】**, confirming these as core ops.

5. Execution Model (State, Recurrence, Saliency)

MORF runs in a **closed loop of perception, interpretation, and action**. At any time, it maintains a modest **semantic state** vector/structure (e.g. intent hypotheses, dialogue context, device context). The processing pipeline for each user utterance (and beyond) is roughly:



1. **Perceptual Processing:** From raw audio/text input, extract relevant features (voice activity, tokens). (This may be part of firmware/codec outside the MORF “brain.”)
2. **Initial Semantic Sketch:** Quickly assemble a *coarse semantic form* of the utterance using the high-priority forms. This is like an artist’s pencil sketch of meaning. It does *not* resolve all details. Instead, it identifies major concepts (e.g. “someone → volume is high”).
3. **Check Sufficiency:** Evaluate if the current sketch is enough to decide on an action or response. Often it will be, especially for clear commands.
4. **Resolve Top Ambiguity:** If not sufficient, select the *most critical unresolved distinction* (based on a salience/priority score) and apply the appropriate forms to clarify it. For example, if both “music” and “background noise” could be loud, resolve which source the user means. This might use a quick learned classifier or a follow-up prompt.
5. **Repeat:** Update the sketch with the resolved info, then loop back to checking sufficiency.
6. **Generate Response/Action:** Once stable, produce an output: either call a firmware API (e.g. `set_volume(...)`) or select an authored/phased response sentence.
7. **State Update:** Store any learning (e.g. adapt a form weight) and incorporate the outcome into memory.

This **adaptive, salience-driven recurrence** means MORF spends most computation on the *informative parts* of dialogue. A simple request like “Down” might skip to action immediately; a tricky metaphor might trigger several refine loops.

State Representation: The “state” might include variables like

[agent, target, intensity, certainty], the dialogue history pointer, and any learned incremental beliefs. We keep this small (maybe a few dozen numbers) so each loop is fast. The S3’s dual cores can allow pipelining (one core listens while the other computes, or parallelizes vector ops).

Salience: Each potential piece of semantic detail is scored by estimated impact. Forms that affect the final decision get higher priority. This mimics attention: only query what matters. (This can be implemented as a small decision tree or even a learned priority network.)

Citation: The idea of *reuse via recurrence* is supported by the S3’s design: its small on-chip cache (64 KB) and fast SIMD make looping cheap [10†L277-L284]. High-speed vector ops make iterative refining (many small GEMMs vs one big network pass) feasible in MCU contexts.

6. Storage Formats

- **Form Library:** The set of forms and their wiring is essentially *the program*. We can encode this as C++ code or a tiny custom VM. Alternatively, a “form grammar” in JSON or binary (e.g. a table of form identifiers and connections). This resides in flash.

- **Learned Parameters:** Any small neural or statistical models (e.g. for word recognition, phrase-to-form mapping) will be quantized and stored as int8 arrays in flash or PSRAM. (ESP-NN expects 8-bit.) E.g. a 100×32 linear layer uses 3.2 KB. We target total learned model size $\lesssim$ 100 KB.
- **Authored Utterances:** High-value canned responses (greetings, confirmations, jokes) are stored as text strings or precompiled code (e.g. phonemes). A few dozen lines of text cost a few hundred bytes.
- **Vocabulary & Embeddings:** If needed, small tables (e.g. 1000 words × 32-dim embeddings = 32 KB) can fit in PSRAM. But we minimize vocabulary by prioritizing necessary words (commands, common objects, device properties).
- **Memory (Log/Knowledge):** A tiny database of recent interactions (maybe last 10 utterances) or learned user preferences (e.g. “preferred volume = 50”). Structured as simple records in external RAM or flash.

All storage is kept compact. We will design a custom format that avoids overhead. For example, forms could be identified by 8-bit IDs instead of text, and the form graph by small indices. We deliberately avoid storing large trees or graphs; if needed, we recompute forms from basic components.

7. Training / Compilation Pipeline

Philosophy: Training is not for omniscience, but for filling the *gaps* in our designed system. We can **autogenerate much content** with LLMs, but we will *structure and filter it by design intent*, not let an LLM define the system.

- **Capability-Driven Scoping:** Start from the device’s capabilities (e.g. volume control, brightness, timers, small talk). From the firmware API list, derive “semantic targets” (Volume, Brightness, etc.) and likely user intents about each (increase, decrease, query). This yields a *semantic schema* and salience priorities.
- **Seed Grammar and Forms:** Manually define a skeletal grammar that covers core intents. For example, an intent “AdjustVolume” with form pattern `[requester] + increase/decrease + (volume | sound)`. This is hand-authored by designers (“we know what the device can do”).
- **LLM-assisted Corpus Generation:** Use an LLM (e.g. GPT) to generate example utterances and paraphrases for each intent/form. For “Volume decrease”, generate dozens of ways a human might ask. (We control the prompt to limit to short chatty style.) Also generate negative examples and ambivalent phrasing to test ambiguity (e.g. “Turn it down” vs “Not too loud”).
- **Human Curation:** Filter the generated corpus to remove noise, and label each utterance with the intended forms/slots. This creates training data for any learned modules (e.g. phrase-to-form classifiers, slot fillers).
- **Learning:** Train only the small components that require it. This might include:
 - A classifier to map raw input (ASR text or acoustic features) to a semantic sketch (which forms to activate, which slots to fill). E.g. a tiny neural net or decision tree.
 - Parameter tuning in forms (e.g. a threshold for “too loud”).

- A language generator for “gap-filler” responses (mini template-based or seq2seq with very small models) for content we can’t author exhaustively.
- **Evaluation Dataset:** We prepare a test set of utterances (both seen and novel) to evaluate coverage and correctness of the system.

This pipeline resembles a “compile” process more than traditional ML training. Many components are rules/templates; learning only refines the parts that benefit from examples. All processing (LLM calls, model training) happens offline on a PC; the ESP32-S3 only gets the resulting compiled data.

8. Evaluation & Metrics

We evaluate MORF on several axes:

- **Functional Accuracy:** For a suite of test utterances, does MORF interpret them correctly (correct intent/forms) and respond appropriately? This is measured by a confusion matrix over intents/slots. Target $\geq 90\%$ on in-domain tests.
- **Semantic Sufficiency:** Does it ask for clarification only when needed? (False-alarm vs miss rates.)
- **Latency:** End-to-end response time on the device (goal: <100 ms per utterance including wake/listen).
- **Resource Usage:** RAM usage (core stack + PSRAM) and Flash footprint. We track total model sizes, SRAM peak. (Goal: fit within 512 KB with margin.)
- **Robustness:** Test under noise (ASR errors) to ensure fallback behavior.
- **User Studies:** Subjective assessment of “naturalness” or “user satisfaction” in short-dialog tasks.

Privacy/Security Testing: Ensure data never leaves the device (no hidden internet calls). Use S3’s secure boot and encryption [22†L263-L268] to protect firmware and any stored user data. Confirm OTA update integrity.

9. Deployment & OTA

We will use standard ESP-IDF tools for deployment:

- **Firmware Images:** Split code into *bootloader*, *firmware (MORF runtime)*, and *data/model* partitions.
- **OTA Mechanism:** Use HTTPS or BLE to push updates. Leverage the S3’s secure boot and flash encryption to only accept signed updates [22†L263-L268] .
- **Model Updates:** Since language evolves, allow *content OTA*: update form grammars or utterance lists without reflashing code. (This could be JSON updated on flash.)
- **Debug/Logging:** Include a lightweight interface (UART or BLE) for diagnostic logging of internal state (for development only).

Care must be taken that an attacker can’t “teach” the device malicious phrases. All OTA channels must authenticate content (e.g. use HMAC or secure key provisioned at manufacturing).

10. Productization Loop

We envision iterative development and use:

1. **Prototype & Internal Test:** Build initial MORF with minimal forms and utterances. Use scripted tests (see Sec.8).
2. **Beta Release:** Ship to a small group. Collect user interactions (anonymized) to find blind spots.
3. **Grade & Improve:** Log failures (where MORF misinterpreted or failed). Add/adjust forms or authored utterances. Possibly expand training corpus for rare cases.
4. **Scale Up:** After validation, prepare for mass production. Ensure manufacturing key burning (for World Controller), finalize apps.
5. **Post-Launch Updates:** Roll out OTA improvements based on field feedback.

For product marketing, emphasize *privacy* (no cloud needed), *reliability* (predictable tiny brain), and *personality* (handcrafted social style). Future versions might add new device capabilities (e.g. new sensors) which feed into new forms.

11. Example Forms & Utterances (Shortlist)

To start, we recommend hand-authoring these core forms and example utterances:

- **Form:** `TARGET_STATE_CHANGE(Device, Property, Direction)`
Description: Express a desired increase/decrease in a device property.
Utterance Examples: "Turn the music down." "Raise the brightness." "Lower the volume a bit."
- **Form:** `FEEDBACK(Confirm/Affirm)`
Description: Generic agreement or acknowledgment.
Utterance Examples: "Okay." "Got it." "Sure thing." "I hear you."
- **Form:** `QUESTION_CONTEXT`
Description: Clarifying or probing.
Examples: "Do you mean this speaker?" "Which device do you mean?" "I'm not sure, please clarify."
- **Form:** `SMALLTALK`
Description: Casual chit-chat unrelated to device control.
Utterance Examples: "Interesting!" "Cool." "I agree." "Haha, thanks."
- **Form:** `PREDICTION_CHECK`
Description: Detect if utterance reflects a prediction or statement.
Examples: "It's a bit loud already." "I think that's fine."
- **Form:** `CONSTRAINT/LEGAL`
Description: Check if a value is out of range.
Examples: "Volume max is 100." "Sorry, that's too high." "I can't do that."

These cover the common “volume” use-case and socializing. As experiments proceed, we’ll identify additional needed forms (e.g. `TIMER_SET`, `ERROR_HANDLING`, etc.) and add utterances. We also include generic fallback phrases like “Sorry, I didn’t get that.”

12. Next Milestones

- **M1 (1 month):** Finalize form list and grammar schema. Prototype a simple form interpreter loop on ESP32-S3.
- **M2 (2 months):** Generate/curate 500+ sample utterances. Train any small classifiers for form detection. Load content on device.
- **M3 (3 months):** Alpha device prototype: test on bench with scripted dialogue. Benchmark latency and memory.
- **M4 (6 months):** Beta test with users. Iterate on failure cases.
- **M5 (9 months):** Security and OTA hardened. Prepare for production.

Each cycle yields a new “branch” of the MORF knowledge: improved forms, reduced error rates, and expanded social phrases.

References: ESP32-S3 datasheet and product info [【6†L228-L232】](#) [【22†L263-L268】](#) ; ESP-NN/ESP-DL performance docs [【10†L277-L284】](#) [【24†L1-L4】](#) . These confirm hardware capabilities (vector DSP, crypto) and optimized ops available on our target MCU. The design above leverages these to achieve “**more cognitive power per byte,**” as intended in the user’s vision.
